Dynamic Exponent-Window Accumulation with Activation Top-2 Selection for Efficient Low-Bit BFP LLM Inference

Ming-Hong Lin
National Taiwan University of Science and Technology
Department of Electrical Engineering
Taipei, Taiwan

Abstract

Block Floating Point (BFP) reduces multiply–accumulate cost by sharing one exponent per block, but partial sums from blocks with different exponents must still be merged in a floating-point accumulator (FP-Acc). As the mantissa width shrinks, the FP-Acc increasingly dominates processing-element power and area, and activation outliers aggravate the problem by widening the exponent spread across blocks. We propose *Dynamic Exponent-Window Accumulation* (DEWA), a lightweight integer accumulator that retains partial sums within a sliding exponent window and forwards only out-of-window events to the FP-Acc, substantially reducing its utilization. To prevent outlier-induced window shifting, we pair DEWA with *Top-2 Activation-only Bi-Exponent* (T2A-BiE) encoding, which gives each activation block a second exponent for at most two outlier lanes while leaving weights in standard single-exponent BFP. Together, the two techniques preserve model accuracy on LLaMA2-7B, LLaMA-3.1-8B, and OPT-6.7B while yielding a smaller and more energy-efficient PE than both a conventional BFP baseline and a recent BFP accelerator design.

CCS Concepts

• **Computer systems organization** → **Neural networks**; • **Hardware** → *Arithmetic and datapath circuits.*

Keywords

Block floating point, large language models, quantization, hardware accelerator, outlier handling

ACM Reference Format:
Ming-Hong Lin. 2026. Dynamic Exponent-Window Accumulation with Activation Top-2 Selection for Efficient Low-Bit BFP LLM Inference. In *63rd ACM/IEEE Design Automation Conference (DAC '26), July 26–29, 2026, Long Beach, CA, USA*. ACM, New York, NY, USA, 6 pages. https://doi.org/XXXXXXX.XXXXXXX

1 Introduction

Low-precision quantization is essential for deploying large language models (LLMs) under tight memory and energy budgets. Power is a first-class constraint in this context: industrial inference accelerators are designed around thermal design power because TDP tracks total cost of ownership, whereas area-based metrics such as performance per square millimeter can look favorable even when energy cost rises [5]. Block Floating Point (BFP) offers a compelling accuracy–cost tradeoff by partitioning tensors into fixed-size blocks, each with a shared exponent, and storing only low-bit mantissas [11, 15]. Recent BFP accelerators exploit this structure to perform intra-block multiply–accumulate (MAC) operations with fixed-point arithmetic, but inter-block partial sums whose shared exponents differ must still be merged in a floating-point accumulator (FP-Acc) [6, 7, 9, 10].

Two coupled bottlenecks limit the efficiency of BFP-based LLM inference. First, the FP-Acc dominates processing-element (PE) cost. As the mantissa width shrinks, the fixed-point multipliers and adder tree become smaller while the FP-Acc stays the same size, so its share of total PE power and area grows (Fig. 2). Second, activation outliers—values whose magnitude far exceeds that of co-located elements—compress the effective mantissa of normal values and widen the exponent spread of partial sums, increasing the frequency at which the FP-Acc must be invoked [3, 8, 12, 16, 17]. These two bottlenecks reinforce each other: the outliers that degrade quantization accuracy also inflate FP-Acc activity, so addressing them jointly is more effective than tackling either in isolation.

This paper develops a hardware-friendly co-design that targets both bottlenecks. Our contributions are:

- **Dynamic Exponent-Window Accumulation (DEWA):** a lightweight integer accumulator that retains partial sums within a sliding exponent window and forwards only out-of-window events to the FP-Acc, substantially reducing its utilization.
- **Activation-only bi-exponent encoding with Top-2 selection (T2A-BiE):** weights remain in single-exponent BFP while each activation block carries a second exponent for at most two outlier lanes, reducing storage and MAC control overhead relative to full bi-exponent encoding [16, 17].
- **End-to-end evaluation:** we validate the proposed W-BFP4 / T2A-BiE4 design point on LLaMA2-7B, LLaMA-3.1-8B, and OPT-6.7B, demonstrating that the combined scheme preserves model accuracy while delivering a PE that is both smaller and more energy-efficient than a conventional BFP baseline and a recent BFP accelerator.

The remainder of this paper reviews BFP deployment challenges (Section 2), presents DEWA, Top-2 selection, and their combination (Section 3), describes the Top-2 index encoding and the DEW-based PE datapath (Section 4), reports experimental results (Section 5), and concludes (Section 6).

DAC '26, Long Beach, CA, USA

ACM ISBN 978-x-xxxx-xxxx-x/YY/MM
https://doi.org/XXXXXXX.XXXXXXX

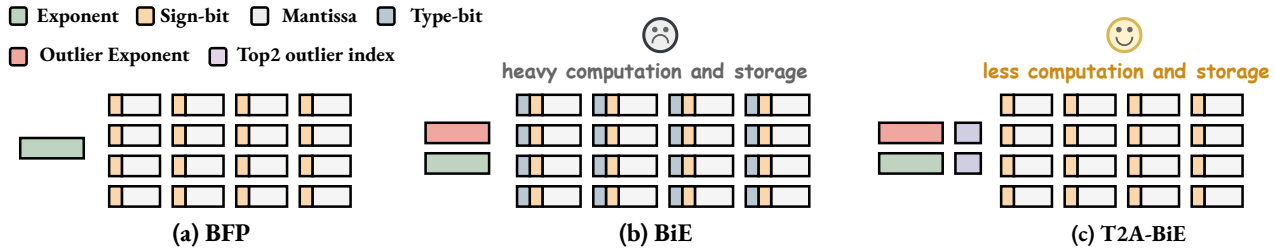

Figure 1: Block-level data formats. (a) Conventional BFP with one shared exponent per block. (b) Bi-exponent BFP (BiE), which adds an outlier exponent and spends a type bit on every element. (c) The proposed Top-2 Activation-only Bi-Exponent BFP (T2A-BiE), which keeps both exponents but replaces the per-element type bits with two outlier indices, so the storage and MAC overhead is lower than that of BiE; weights remain single-exponent BFP as in (a).

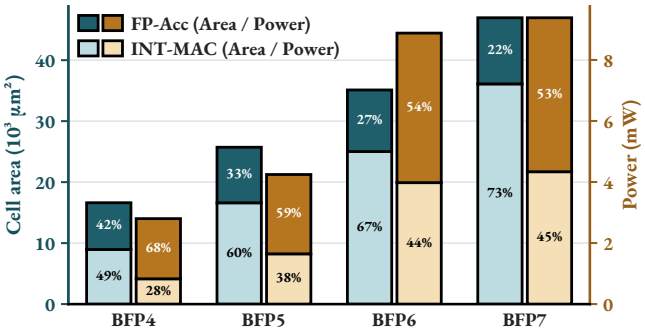

Figure 2: Component-level power and area of a synthesized baseline BFP-PE (TSMC 90 nm, 100 MHz). Power is measured under LLaMA2-7B's real activation distribution.

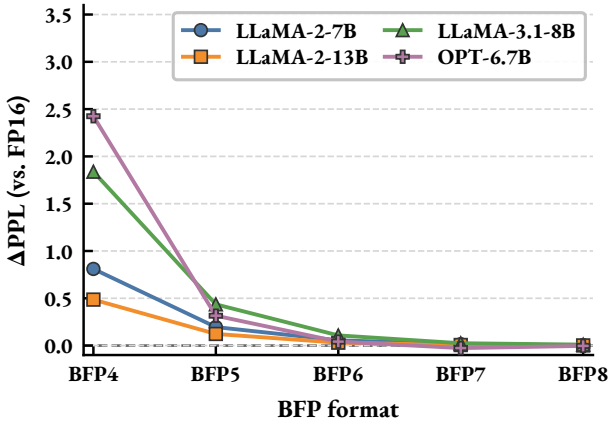

Figure 3: WikiText-2 ΔPPL of vanilla BFP versus FP16 as the mantissa width decreases (group size 16).

2 Background

2.1 Block Floating Point

Low-precision integer quantization reduces storage and arithmetic cost, but a single tensor-level scaling factor can be dominated by rare large-magnitude values. This issue is particularly severe in LLM activations, where a small number of outliers may compress most nonoutlier values into a limited set of quantization levels and degrade low-bit accuracy [2, 4, 13, 14].

Block Floating Point (BFP) provides a more fine-grained accuracy–cost tradeoff by partitioning tensors into fixed-size blocks and assigning an independent shared exponent to each block. Each element stores only a sign and a low-bit mantissa (Fig. 1(a)), while the exponent overhead is amortized across the block. Intra-block computation requires only fixed-point multiply–accumulate operations, enabling BFP to achieve higher accuracy at a cost comparable to integer quantization [1, 11, 15].

2.2 Deployment Challenges

Applying BFP to LLM inference acceleration introduces two major bottlenecks.

(1) Inter-block floating-point accumulation. A state-of-the-art BFP processing element (BFP-PE) can perform MAC operations within each block using low-bit fixed-point arithmetic. However, shared exponents differ across blocks, so partial sums cannot be directly accumulated in the integer domain. An FP accumulator (FP-Acc) is therefore required to perform inter-block exponent alignment and floating-point accumulation. The FP-Acc typically includes alignment, normalization, addition, and fixed-point-to-floating-point conversion logic. In synthesized low-bit BFP-PE configurations, the FP-Acc accounts for up to 68% of PE power and 42% of its area when the PE is driven by LLaMA2-7B's real activation distribution (Fig. 2), making it a primary obstacle to improving the energy and area efficiency of BFP accelerators [6, 7, 9].

(2) Activation outliers. BFP confines the influence of an outlier to its local block but does not eliminate the outlier itself. When a block contains a large-magnitude outlier, the shared exponent is determined by that value, and nonoutlier values in the same block lose a substantial number of effective mantissa bits [3, 8, 12, 16, 17]. This loss becomes more severe as the mantissa width decreases (Fig. 3).

These two bottlenecks are coupled. Outliers degrade the precision of nonoutlier values during quantization and can also widen the exponent distribution of resulting partial sums, increasing the likelihood that inter-block updates require the FP-Acc. Consequently,

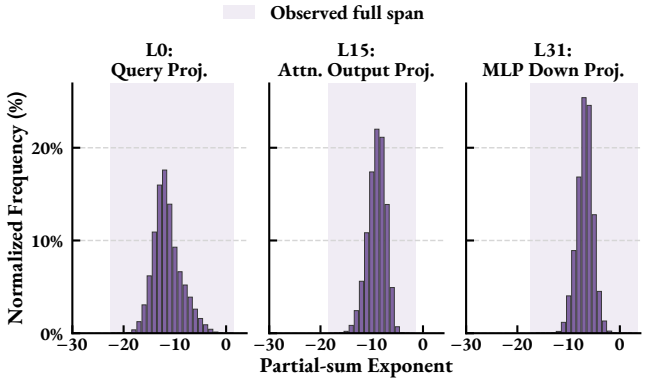

Figure 4: BFP4 partial-sum exponent distributions on three LLaMA2-7B layers (group size 16). The shaded band is the observed full span.

effective outlier handling has the potential to improve numerical fidelity while simultaneously reducing FP-Acc activity. This observation motivates the approach developed in this work.

2.3 Bi-Exponent BFP for Outlier Separation

A common remedy for the precision loss described above is to give outliers their own exponent instead of forcing an entire block to share one. BiE [17] and Oiso [16] store two shared exponents per block (BiE format), one for nonoutlier values and one for outliers, and prepend a type bit to each element that selects which exponent applies (Fig. 1(b)). Nonoutlier values therefore keep their effective mantissa bits even when the block contains a large-magnitude value.

The accuracy gain, however, comes with non-trivial overhead. Because outliers are inherently rare, a per-element type bit spends storage on every element to describe a small minority, and the two exponents must be applied to both operands of a MAC, yielding four exponent-pair cases that the datapath has to support. This overhead is what makes a direct combination of bi-exponent BFP with a low-cost accumulator unattractive, and it is the starting point for the format we propose in Section 3 (Fig. 1(c)).

3 Proposed Method

3.1 Dynamic Exponent-Window Accumulation

We observed the distribution of partial sums in LLaMA2-7B (Fig. 4). The distribution remains highly concentrated, indicating that a low-cost accumulator with a narrow dynamic range can be introduced prior to forwarding data to the FP-Acc.

Specifically, we define an accumulation window

$$[E_{\max} - T_{\text{drop}},\ E_{\max} + T_{\text{replace}}] \tag{1}$$

where $E_{\max}$ denotes the exponent of the current maximum partial sum, and T_{drop} and T_{replace} are the drop and replace thresholds. When a new partial sum arrives:

- if its exponent falls below the lower bound, it is discarded;
- if it exceeds the upper bound, it directly replaces the value currently held in the accumulator;
- otherwise, it is accumulated using the low-cost accumulator.

Table 1: Outlier occupancy per G16 activation block (W-BFP4/A-BiE4, LLaMA2-7B).

Outliers per block	Block count	Block percentage
0	21,437,737,318	**88.60%**
1	2,373,974,147	**9.81%**
2	241,189,169	**1.00%**
≥ 3	141,941,990	0.59%

This approach substantially reduces FP-Acc utilization and thereby lowers power consumption [7, 9]. We refer to this method as **Dynamic Exponent-Window Accumulation (DEWA)**.

Although the majority of partial sums are concentrated, the overall distribution still exhibits a large dynamic range. We attribute this to the influence of outliers, which cause the dynamic exponent window to shift significantly and thereby induce precision loss. We therefore argue that outlier separation is essential as a complement to DEWA. By separating activation outliers, a substantially higher proportion of partial sums fall within the accumulation window.

3.2 Activation Top-2 Selection

Outlier separation is therefore required, but the bi-exponent format of Section 2.3 cannot be dropped into DEWA as is: its per-element type bits and four MAC cases add cost that the rare occurrence of outliers does not justify. We instead retain the two exponents and remove the overhead through two changes:

(1) Since the majority of outliers concentrate on specific activation channels, applying bi-exponent encoding to weights yields benefits that do not justify the associated overhead. We therefore adopt single-exponent BFP for weights while retaining bi-exponent encoding for activations.
(2) We further examined the number of outliers occurring within each block (Table 1). 99.4% of G16 activation blocks contain at most two outliers. We propose **Activation Top-2 Selection**, wherein at most the two largest outliers are selected per block, further reducing hardware overhead.

We call the resulting activation encoding **Top-2 activation-only bi-exponent BFP (T2A-BiE)**, shown in Fig. 1(c): the two indices identify which lanes of the G16 block use the outlier exponent, so no element carries a type bit. Paired with 4-bit mantissas, the full configuration is denoted W-BFP4 / T2A-BiE4.

Activation-only BiE and Top-2 capping have negligible impact on LLaMA2-7B perplexity relative to full BiE (Fig. 5).

3.3 Hybrid Top-2 DEWA

Top-2 keeps activation outliers from shifting $E_{\max}$, which is what DEWA needs to keep a narrow window. We apply the two together and call the result **Hybrid Top-2 DEWA**: W-BFP4 / T2A-BiE4 encoding with the window of Eq. (1) on the normal accumulator.

A full 2-D sweep over every $(T_{\text{drop}}, T_{\text{replace}})$ pair is more than we need to pick a design point. Fig. 6 shows a representative slice: the symmetric corners $(8, 8)$ and $(10, 10)$, and a $T_{\text{drop}} = 9$ cut with T_{replace} from 9 down to 2. The best pair is not the same on every model. LLaMA2-7B and LLaMA-3.1-8B stay close to T2A-BiE from $(9, 9)$ down to $(9, 3)$ and rise at $(9, 2)$; both get worse at $(8, 8)$,

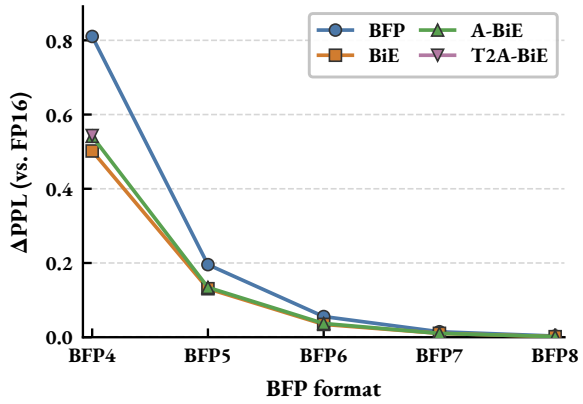

Figure 5: WikiText-2 ΔPPL versus FP16 on LLaMA2-7B (G16). T2A-BiE stays close to full BiE and activation-only BiE.

LLaMA-3.1-8B by a large margin. OPT-6.7B is lowest at $(8, 8)$, where Hybrid Top-2 DEWA falls below T2A-BiE.

Reducing T_{replace} shortens the integer window and cuts the dynamic range of the low-cost accumulator, which saves area. This paper still uses one pair, $(9, 3)$, for the hardware-cost analysis. It is the smallest T_{replace} on the $T_{\text{drop}} = 9$ cut that all three models keep near T2A-BiE, and Section 4 implements the hybrid PE at $(T_{\text{drop}}, T_{\text{replace}}) = (9, 3)$.

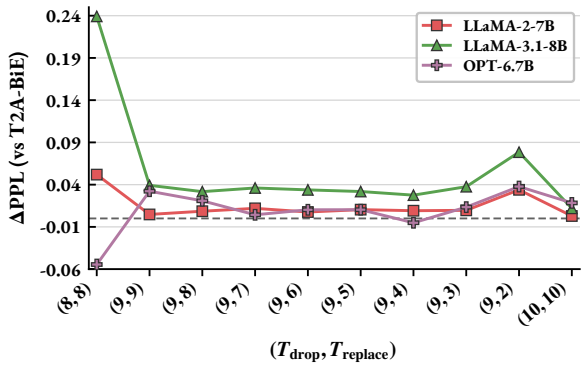

Figure 6: WikiText-2 ΔPPL of Hybrid Top-2 DEWA versus T2A-BiE (G16) on LLaMA2-7B, LLaMA-3.1-8B, and OPT-6.7B. The best $(T_{\text{drop}}, T_{\text{replace}})$ pair varies by model; the hardware analysis uses $(9, 3)$.

4 Hardware Architecture

4.1 Top-2 Activation BiE Index Encoding

T2A-BiE keeps two shared exponents per activation block, $E_{\text{a}}^{(\text{n})}$ for normal values and $E_{\text{a}}^{(\text{o})}$ for outliers, while weights stay in single-exponent BFP. Instead of a type bit on every element, two outlier indices name the lanes that use $E_{\text{a}}^{(\text{o})}$ (Fig. 1(c)). Each index is 4 bits at group size 16, so a block carries 8 index bits rather than 16 type bits. The pair $(\text{OI}_1, \text{OI}_2)$ encodes how many outliers a block holds and where they are, so no separate count field is needed (Table 2).

Table 2: Top-2 outlier-index encoding for a G16 activation block.

Outliers per block	OI_1	OI_2
0	15 (reserved)	15 (reserved)
1	lane index i	0 (unused)
2	lane index i_1	lane index i_2

The pair $(15, 15)$ is reserved for an empty block, so a single outlier on lane 15 is stored as $(15, 0)$. Since $\text{OI}_2 = 0$ already marks the second slot as unused, the encoder never writes lane 0 into OI_2; a lane-0 outlier always occupies OI_1, and two outliers keep $i_1 \neq i_2$. Decoding then costs two comparisons: whether both indices are all ones, and whether OI_2 is nonzero.

Because weights carry a single exponent, every product in a block lands on one of two exponent pairs, $E_{\text{w}} + E_{\text{a}}^{(\text{n})}$ or $E_{\text{w}} + E_{\text{a}}^{(\text{o})}$, rather than the four cases of full BiE. The PE therefore never decodes a per-element type bit; it only moves the lanes named by OI_1 and OI_2 to fixed outlier positions, which is what the dispatcher of Section 4.3 implements.

4.2 DEW-Based PE

The PE takes one weight block and one activation block, the latter carrying the two outlier indices of Section 4.1. All 16 lanes are multiplied in fixed point, and the indices then split the results into two paths: the normal lanes enter an adder tree that reduces them to one partial sum per block, while each named outlier lane stays separate and never joins the tree.

The normal partial sum goes to the DEW accumulator, which accumulates in the integer domain inside the exponent window of Eq. (1); only results that fall outside the window spill to the FP-Acc. An outlier partial sum carries the other shared exponent and is therefore sent straight to the FP-Acc. Each path travels with its own block exponent, $E_{\text{w}} + E_{\text{a}}^{(\text{n})}$ or $E_{\text{w}} + E_{\text{a}}^{(\text{o})}$, and the two merge in a single FP-Acc that produces the output (Fig. 7(a)). The FP-Acc thus handles only outliers and the few out-of-window updates instead of every inter-block partial sum.

4.3 Outlier Dispatcher and Adder Tree

Handling an outlier where it sits would require every node of the adder tree to be bypassable, which is expensive to control. We move the data instead: guided by OI_1 and OI_2, a dispatcher exchanges the two outlier lanes with lanes 15 and 14 and leaves the remaining lanes in place (Fig. 7(b)). After the exchange the outliers always occupy the same two positions, so no downstream logic has to read the indices again. This is a pairwise swap rather than a full crossbar, costing one small multiplexer per lane.

The first level of the 16-lane adder tree has eight nodes. Seven of them always add a fixed pair of adjacent lanes; the eighth covers lanes 14 and 15, the positions the outliers were moved to, and changes role with the outlier count:

- With no outlier it adds lanes 14 and 15 as usual, and the tree behaves like that of a conventional BFP-PE.
- With one outlier it passes only lane 14 into the tree, while the product on lane 15 leaves as the outlier partial sum.

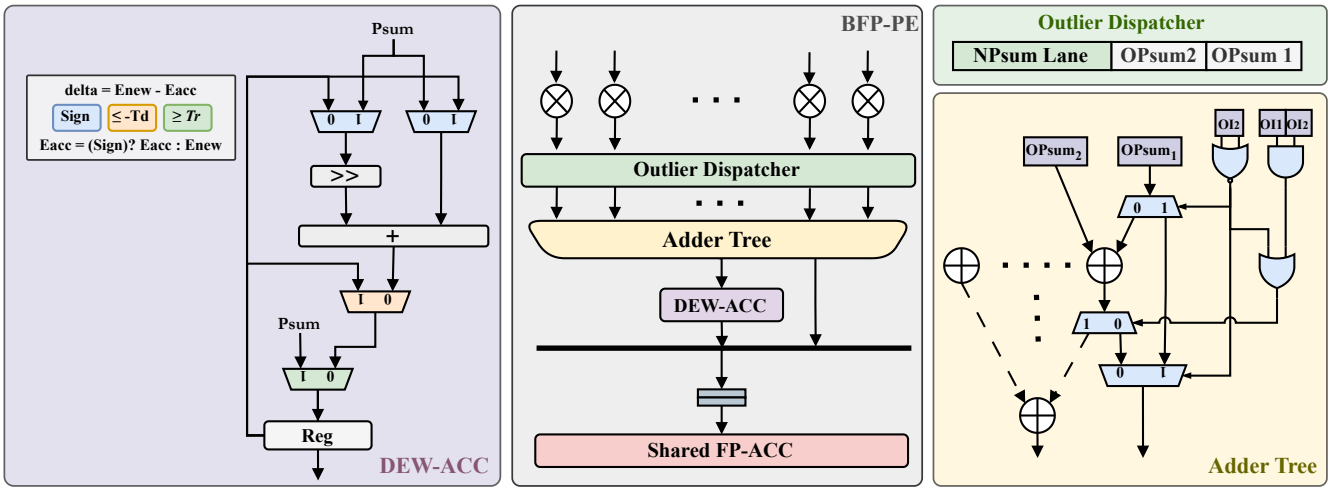

Figure 7: Proposed BFP-PE for W-BFP4 / T2A-BiE4 (group size 16). (a) Overall PE, where the outlier indices split each block into a normal DEWA path and up to two outlier lanes that share one FP-Acc. (b) Outlier dispatcher and the reconfigurable last node of the first adder-tree level. (c) DEW accumulator with its single shifter and exponent register.

- With two outliers it adds lanes 14 and 15, and the result bypasses the tree and goes to the FP-Acc.

One adder therefore serves all three cases, and no separate adder is spent on outliers.

4.4 DEW Accumulator

The accumulator holds two pieces of state: one integer accumulator and the exponent E_{acc} it currently refers to (Fig. 7(c)). Each arriving partial sum is compared against E_{acc}, and the exponent difference decides whether Eq. (1) drops, replaces, or accumulates it. The three outcomes are three inputs of the same multiplexer, so they do not need separate datapaths.

Only the accumulate case needs alignment. The operand with the smaller exponent is the one shifted right, while the other bypasses the shifter and enters the adder directly, and the accumulator adopts the larger exponent. A single shifter therefore serves the whole accumulator, with the sign of the exponent difference selecting whether it acts on the incoming partial sum or on the stored value.

Two events hand a result to the FP-Acc: the integer accumulator overflowing, and the flush at the end of a dot product. On an overflow the accumulator is cleared and restarts from the next partial sum. The number of FP-Acc requests therefore follows the number of overflows rather than the number of blocks, which is where the reduction in FP-Acc utilization comes from.

5 Evaluation

5.1 Determining the Optimized DEW-Acc Setup

The DEWA window is already fixed at $(T_{\text{drop}}, T_{\text{replace}}) = (9, 3)$. The remaining hardware parameter is the bit-width of the integer accumulator. A wider DEW-Acc overflows less often and therefore issues fewer FP-Acc requests on the normal path, but the adder, shifter, and register all grow with the width.

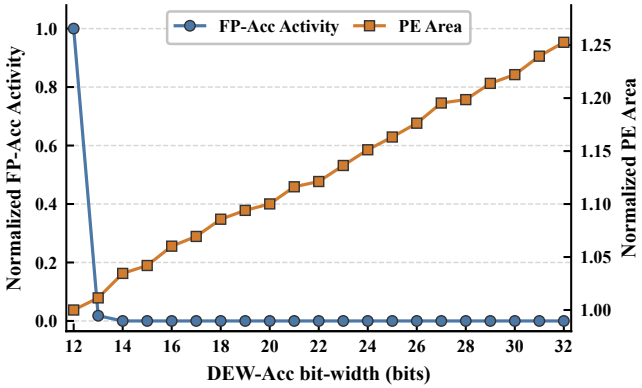

Figure 8: Normalized FP-Acc overflow activity and synthesized PE area versus DEW-Acc bit-width on LLaMA2-7B (W-BFP4 / T2A-BiE4, G16, $(T_{\text{drop}}, T_{\text{replace}}) = (9, 3)$). Both series are normalized to W12.

We sweep 12–32 bits on LLaMA2-7B under W-BFP4 / T2A-BiE4 (G16), using one frozen activation stream so that the width is the only variable. FP-Acc activity here is the overflow-event count; outlier requests do not depend on the width and are left out of this curve. PE area is the synthesized area of the full PE at each width.

Fig. 8 shows both series normalized to W12. W13 is the better choice: overflow activity drops sharply while PE area barely grows, so we use a 13-bit DEW accumulator.

5.2 Hardware and Energy Comparison

We compare three synthesized PEs in TSMC 90 nm at 100 MHz: a conventional BFP4 baseline, a Bucket Getter PE [9], and the proposed PE with the 13-bit DEW accumulator of Section 5.1. Fig. 9 reports cell area and energy, each normalized to the baseline.

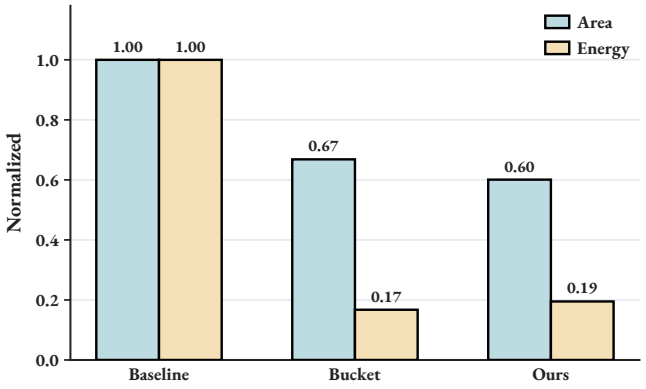

Figure 9: Normalized PE area and energy of a BFP4 baseline, a Bucket Getter PE, and the proposed PE (TSMC 90 nm, 100 MHz). Both metrics are normalized to the baseline.

The proposed PE is smaller than both the baseline and Bucket Getter. Its energy is close to that of Bucket Getter and well below the baseline.

6 Conclusion

This paper presented a co-design of data format and accumulation strategy for efficient low-bit BFP inference of large language models. Dynamic Exponent-Window Accumulation (DEWA) introduces a lightweight integer accumulator that absorbs the majority of inter-block partial sums within a narrow exponent window, forwarding only out-of-window events to the floating-point accumulator. Top-2 Activation-only Bi-Exponent encoding (T2A-BiE) complements DEWA by separating up to two outlier lanes per activation block with a second shared exponent, preventing outliers from shifting the accumulation window while incurring less storage and control overhead than full bi-exponent schemes.

Experiments on LLaMA2-7B, LLaMA-3.1-8B, and OPT-6.7B show that the combined W-BFP4 / T2A-BiE4 configuration preserves model accuracy relative to strong baselines. Synthesis results in TSMC 90 nm confirm that the proposed PE is both smaller and more energy-efficient than a conventional BFP baseline and a Bucket Getter PE.

Future work includes extending the evaluation to larger models and additional tasks beyond language modeling, exploring adaptive window parameters that vary across layers, and integrating the proposed PE into a full systolic-array accelerator to assess system-level throughput and energy.

References

[1] C. Fang, M. Shi, R. Geens, A. Symons, Z. Wang, and M. Verhelst. 2025. Anda: Unlocking Efficient LLM Inference with a Variable-Length Grouped Activation Data Format. In *Proc. IEEE Int. Symp. High-Performance Computer Architecture (HPCA)*. 1467–1481. doi:10.1109/HPCA61900.2025.00110

[2] C. Guo et al. 2023. OliVe: Accelerating Large Language Models via Hardware-Friendly Outlier-Victim Pair Quantization. In *Proc. 50th Annu. Int. Symp. Computer Architecture (ISCA)*. 1–15. doi:10.1145/3579371.3589038

[3] X. Han et al. 2025. BBAL: A Bidirectional Block Floating Point-Based Quantisation Accelerator for Large Language Models. In *Proc. 62nd ACM/IEEE Design Automation Conf. (DAC)*. 1–7.

[4] W. Huang et al. 2024. BiLLM: Pushing the Limit of Post-Training Quantization for LLMs. In *Proc. 41st Int. Conf. Machine Learning (ICML) (PMLR, Vol. 235)*. 20023–20042.

[5] N. P. Jouppi et al. 2021. Ten Lessons from Three Generations Shaped Google's TPUv4i: Industrial Product. In *Proc. 48th Annu. Int. Symp. Computer Architecture (ISCA)*. 1–14. doi:10.1109/ISCA52012.2021.00010

[6] X. Ju et al. 2025. SmartBlock: Adaptive Block Floating Point Quantization for Efficient DNN Acceleration. In *Proc. 54th Int. Conf. Parallel Processing (ICPP)*. 428–438. doi:10.1145/3754598.3754660

[7] X. Ju et al. 2025. WinAcc: Window-Based Acceleration of Neural Networks Using Block Floating Point. In *Proc. Design, Automation & Test in Europe Conf. (DATE)*. 1–7. doi:10.23919/DATE64628.2025.10992780

[8] D. Li, Z. Li, B. Liu, and J. Wu. 2025. Improving Efficiency in Large Language Models via Extendable Block Floating Point Representation. In *Findings Assoc. Comput. Linguistics: ACL 2025*. 14861–14873. doi:10.18653/v1/2025.findings-acl.768

[9] Y.-C. Lo and R.-S. Liu. 2023. Bucket Getter: A Bucket-Based Processing Engine for Low-Bit Block Floating Point (BFP) DNNs. In *Proc. 56th IEEE/ACM Int. Symp. Microarchitecture (MICRO)*. 1002–1015. doi:10.1145/3613424.3614249

[10] V. Natesh, H. T. Kung, and D. Kong. 2025. MGS: Markov Greedy Sums for Accurate Low-Bitwidth Floating-Point Accumulation. *arXiv preprint arXiv:2504.09072* (2025). doi:10.48550/arXiv.2504.09072

[11] B. D. Rouhani et al. 2020. Pushing the Limits of Narrow Precision Inferencing at Cloud Scale with Microsoft Floating Point. In *Advances in Neural Information Processing Systems 33 (NeurIPS)*.

[12] X. Wang, J. Li, Y. Sun, and W. He. 2026. Harmonia: Algorithm-Hardware Co-Design for Memory- and Compute-Efficient BFP-Based LLM Inference. *arXiv preprint arXiv:2602.04595* (2026). doi:10.48550/arXiv.2602.04595

[13] G. Xiao, J. Lin, M. Seznec, H. Wu, J. Demouth, and S. Han. 2023. SmoothQuant: Accurate and Efficient Post-Training Quantization for Large Language Models. In *Proc. 40th Int. Conf. Machine Learning (ICML) (PMLR, Vol. 202)*. 38087–38099.

[14] A. H. Zadeh, I. Edo, O. M. Awad, and A. Moshovos. 2020. GOBO: Quantizing Attention-Based NLP Models for Low Latency and Energy Efficient Inference. In *Proc. 53rd IEEE/ACM Int. Symp. Microarchitecture (MICRO)*. 811–824. doi:10.1109/MICRO50266.2020.00071

[15] S. Q. Zhang, B. McDanel, and H. T. Kung. 2022. FAST: DNN Training Under Variable Precision Block Floating Point with Stochastic Rounding. In *Proc. IEEE Int. Symp. High-Performance Computer Architecture (HPCA)*. 846–860. doi:10.1109/HPCA53966.2022.00067

[16] L. Zou et al. 2026. Oiso: Outlier-Isolated Data Format for Low-Bit Large Language Model Quantization. *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.* 45, 2 (Feb. 2026), 929–942. doi:10.1109/TCAD.2025.3585023

[17] L. Zou, W. Zhao, S. Yin, C. Bai, Q. Sun, and B. Yu. 2024. BiE: Bi-Exponent Block Floating-Point for Large Language Models Quantization. In *Proc. 41st Int. Conf. Machine Learning (ICML) (PMLR, Vol. 235)*. 62978–62992.